

Graph Representations (Adjacency Matrix & Adjacency List) — Data Structures

Definition: Graph representations are memory data structures used to store the vertices and edge relationships of a graph in computer memory. The two primary methods are **Adjacency Matrix** and **Adjacency List**.

1. Detailed Technical Explanation

Sample Graph:

```
(0) ---- (1)
 |   \   |
 |   \   |
 |   \   |
(3) ---- (2)
```

1. Adjacency Matrix Representation

A 2D array `adj[V][V]` of size $V \times V$, where `adj[i][j] = 1` if an edge exists from vertex `i` to vertex `j`, and `0` otherwise.

Adjacency Matrix for Sample Graph:

	0	1	2	3
0	0	1	1	1
1	1	0	1	0
2	1	1	0	1
3	1	0	1	0

- **Pros:** Fast edge existence query in $O(1)$ time (`adj[u][v] == 1`).
- **Cons:** High memory footprint $O(V^2)$, inefficient for sparse graphs.

2. Adjacency List Representation

An array of linked lists `adj[V]`, where each element `adj[i]` points to a linked list of neighboring vertices connected to vertex `i`.

Adjacency List for Sample Graph:

```
[0] -> [1] -> [2] -> [3] -> NULL
[1] -> [0] -> [2] -> NULL
[2] -> [0] -> [1] -> [3] -> NULL
[3] -> [0] -> [2] -> NULL
```

- **Pros:** Memory efficient $O(V + E)$ for sparse graphs; iterates over neighbors in $O(deg(u))$ time.
- **Cons:** Checking edge existence between **u** and **v** takes $O(deg(u))$ time.

2. Complete Comparison Table

Feature	Adjacency Matrix	Adjacency List
Space Complexity	$O(V^2)$ (Fixed)	$O(V + E)$ (Dynamic)
Check Edge (u, v)	$O(1)$ (Instant index lookup)	$O(deg(u))$ (List traversal)
Find All Neighbors	$O(V)$ (Scan row)	$O(deg(u))$ (Traverse list)
Add / Delete Vertex	$O(V^2)$ (Resize matrix)	$O(1)$ (Append list head)
Best Suited For	Dense Graphs (**)	E

3. C Implementation of Adjacency List

```
#include <stdio.h>
#include <stdlib.h>

struct AdjListNode {
    int dest;
    struct AdjListNode* next;
};

struct Graph {
    int V;
    struct AdjListNode** array;
};

struct Graph* createGraph(int V) {
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
    graph->V = V;
    graph->array = (struct AdjListNode**)malloc(V * sizeof(struct AdjListNode));
    for (int i = 0; i < V; ++i)
        graph->array[i] = NULL;
    return graph;
}

void addEdge(struct Graph* graph, int src, int dest) {
    // Add edge from src to dest
    struct AdjListNode* newNode = (struct AdjListNode*)malloc(sizeof(struct AdjListNode));
    newNode->dest = dest;
    newNode->next = graph->array[src];
    graph->array[src] = newNode;

    // For undirected graph, add dest to src
    newNode = (struct AdjListNode*)malloc(sizeof(struct AdjListNode));
    newNode->dest = src;
    newNode->next = graph->array[dest];
    graph->array[dest] = newNode;
}
```

4. Must-Write Points for Exams

- In an undirected graph's adjacency matrix, the matrix is always **symmetric** about the main diagonal (`adj[i][j] == adj[j][i]`).
- Adjacency list saves significant memory when representing real-world sparse networks (road maps, web pages, social graphs).

5. Quick Recall Flow

Adj Matrix: 2D Array $V \times V$, $O(V^2)$ Space, $O(1)$ Edge Lookup | Adj List: Array of Linked Lists, $O(V+E)$ Space, Best for Sparse Graphs

